# Kafka Streaming Integration

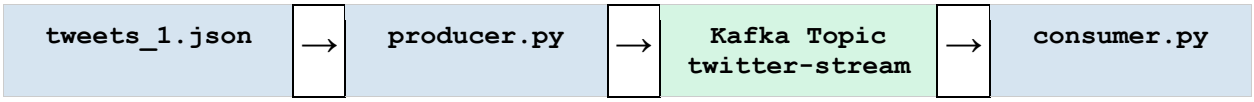
Team Documentation — Twitter Real-Time Data Engineering Pipeline

---

## 1. Project Overview

This document covers the Kafka Streaming & Integration work completed by Team 2 as part of the Twitter Real-Time Data Engineering Pipeline project. The pipeline streams tweet data from a local JSON dataset through Apache Kafka, simulating a live Twitter/X data stream.

### Pipeline Architecture




## 2. Prerequisites

### Software Requirements

| Component | Version | Purpose |
|---|---|---|
| Apache Kafka | 3.x or later | Message broker for streaming |
| Apache Zookeeper | Bundled with Kafka | Kafka cluster coordination |
| Python | 3.8+ | Running producer & consumer scripts |
| kafka-python | Latest (pip) | Python Kafka client library |
| Java (JDK) | 11 or later | Required to run Kafka/Zookeeper |

### Python Library Installation

```
pip install kafka-python
```

## 3. Kafka Setup & Configuration

### Step 1 — Download & Extract Kafka

Download Apache Kafka from https://kafka.apache.org/downloads and extract to your preferred directory (e.g., C:\kafka on Windows or /opt/kafka on Linux/Mac).

## Step 2 — Start Zookeeper

Open a terminal and run the following command from the Kafka root directory:

**Windows:**

```
bin\windows\zookeeper-server-start.bat config\zookeeper.properties
```

**Linux / macOS:**

```
bin/zookeeper-server-start.sh config/zookeeper.properties
```

## Step 3 — Start Kafka Broker

Open a second terminal window and run:

**Windows:**

```
bin\windows\kafka-server-start.bat config\server.properties
```

**Linux / macOS:**

```
bin/kafka-server-start.sh config/server.properties
```

## Step 4 — Create the Kafka Topic

Open a third terminal and run the following command to create the topic used by this pipeline:

**Windows:**

```
bin\windows\kafka-topics.bat --create --topic twitter-stream --
bootstrap-server localhost:9092 --partitions 1 --replication-factor 1
```

**Linux / macOS:**

```
bin/kafka-topics.sh --create --topic twitter-stream --bootstrap-server
localhost:9092 --partitions 1 --replication-factor 1
```

Verify the topic was created:

```
bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

# 4. Project Files

| File | Description |
|---|---|
| tweets_1.json | Sample dataset — 448,093 labelled tweets (tweet_id, entity, sentiment, tweet) |
| producer.py | Reads tweets from JSON and sends them to the Kafka topic one by one |
| consumer.py | Listens to the Kafka topic and prints each received tweet to the terminal |

### Tweet Data Structure

Each tweet record in tweets_1.json follows this JSON schema:

```
{
    "tweet_id":  2401,
    "entity":    "Borderlands",
    "sentiment": "Positive",
    "tweet":     "Borderlands 2 is so good omg"
}
```

# 5. Running the Pipeline

### Step 1 — Run the Producer

Place tweets_1.json in the same directory as producer.py, then run:

```
python producer.py
```

The producer will read each tweet and send it to the twitter-stream topic with a 2-second delay between messages. You will see output like:

```
Sent: {'tweet_id': 2401, 'entity': 'Borderlands', 'sentiment':
'Positive', 'tweet': '...'}
```

### Step 2 — Run the Consumer

In a separate terminal window, run:

```
python consumer.py
```

The consumer will connect to the topic and print each received message in real time:

```
Waiting for messages...
Received: {'tweet_id': 2401, 'entity': 'Borderlands', 'sentiment':
'Positive', 'tweet': '...'}
```

## 6. Script Reference

### producer.py

| Parameter | Value | Description |
|---|---|---|
| bootstrap_servers | localhost:9092 | Kafka broker address |
| value_serializer | json.dumps + encode UTF-8 | Serializes Python dict to JSON bytes |
| topic | twitter-stream | Kafka topic messages are sent to |
| time.sleep(2) | 2 seconds | Delay between each tweet sent |

### consumer.py

| Parameter | Value | Description |
|---|---|---|
| bootstrap_servers | localhost:9092 | Kafka broker address |
| auto_offset_reset | earliest | Read from the beginning if no prior offset |
| value_deserializer | json.loads + decode UTF-8 | Deserializes JSON bytes to Python dict |
| topic | twitter-stream | Kafka topic to subscribe to |

## 7. Team Deliverables

| Deliverable | Status | Notes |
|---|---|---|
| Kafka installation & configuration | Complete | Zookeeper + Kafka broker running on localhost:9092 |
| Kafka topic creation (twitter-stream) | Complete | 1 partition, replication factor 1 |
| producer.py | Complete | Streams 448,093 tweets with 2s delay |
| consumer.py | Complete | Reads and prints all incoming messages |
| Sample dataset (tweets_1.json) | Complete | 448,093 records across multiple entities |

| Deliverable | Status | Notes |
|---|---|---|
| Streaming logs / screenshots | Complete | Producer and Consumer terminal outputs captured |
| Kafka integration documentation | Complete | This document |

## 8. Troubleshooting

| Issue | Cause | Fix |
|---|---|---|
| Connection refused on port 9092 | Kafka broker not started | Start Zookeeper first, then Kafka |
| Topic not found error | Topic was not created | Run the kafka-topics.sh --create command |
| FileNotFoundError: tweets.json | Wrong filename in producer.py | Change to tweets_1.json on line 9 |
| No messages in consumer | auto_offset_reset issue | Restart consumer before producer, or set offset to earliest |
| kafka-python not found | Library not installed | Run: pip install kafka-python |